Large Language Models
Homework 1

Chia-Hsuan Hsu Student ID: 315511079

Fall 2026

1 N-Gram, RNN, and LSTM Models for Language Identification

1.1 Setup

The course-provided splits are used without modification: 1,120 training, 140 validation, and 140 test texts per language (5,600 / 700 / 700 in total). A shared character vocabulary of size $|V| = 129$ is built from `train.csv` only, including the special tokens `<PAD>`, `<UNK>`, `<BOS>`, and `<EOS>`. Characters that never occur in the training data are mapped to `<UNK>`. All models use the same vocabulary and the same special-token policy: `<BOS>` and `<PAD>` are never prediction targets, while `<EOS>` is. Every text is assigned to the language whose model gives the lowest perplexity,

$$\hat{l} = \arg\min_{l \in \mathcal{L}} \mathrm{PPL}(x \mid l),$$

and all validation and test perplexities are corpus-level: the total negative log-likelihood under the ground-truth language model divided by the total number of prediction targets. Per-text perplexities are never averaged.

1.2 Character-Level Bigram and Trigram Models

For each language, bigram and trigram counts are collected from its training texts and smoothed with Add-α ($\alpha = 1$):

$$p(c_t \mid c_{t-1}, l) = \frac{C_l(c_{t-1}, c_t) + \alpha}{C_l(c_{t-1}) + \alpha|V|}, \qquad p(c_t \mid c_{t-2}, c_{t-1}, l) = \frac{C_l(c_{t-2}, c_{t-1}, c_t) + \alpha}{C_l(c_{t-2}, c_{t-1}) + \alpha|V|}.$$

Each text is left-padded with $n-1$ `<BOS>` tokens. As a result, the first character is also predicted, and both models score exactly the same targets: every character plus `<EOS>`.

Table 1: Results of the n-gram models.

Model	Val PPL	Test PPL	Accuracy	Macro-F1	Build time (s)	Inference time (s)
Bigram	11.4373	11.3264	0.9729	0.9727	1.913	0.166
Trigram	**9.7008**	**9.4022**	**0.9871**	**0.9871**	2.120	0.184

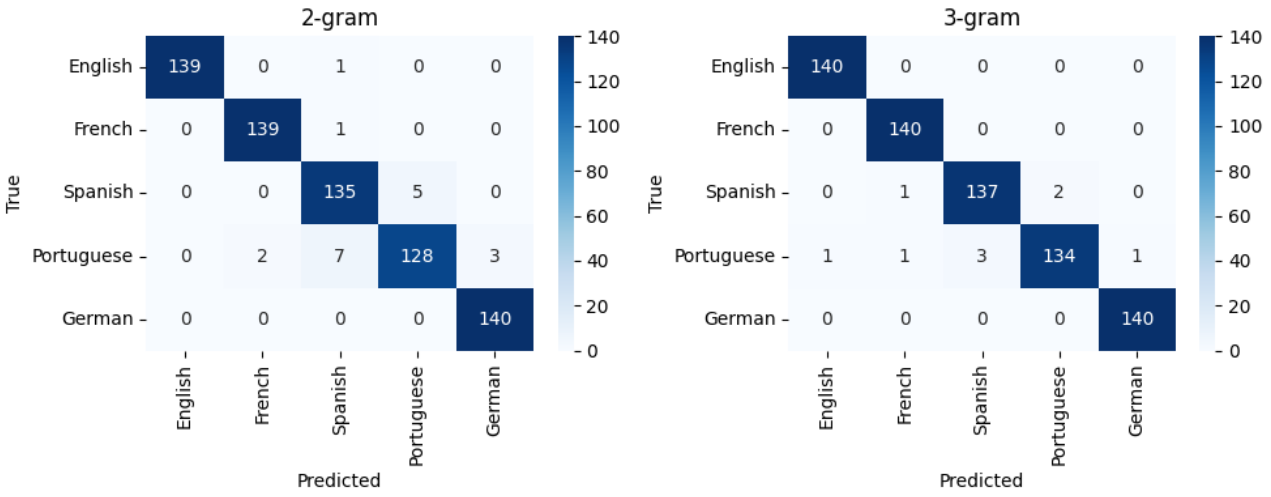


Figure 1: Test confusion matrices of the bigram (left) and trigram (right) models.

Results. The trigram model is better than the bigram model on every metric. Its validation perplexity is 9.70 instead of 11.44, and its test accuracy is 0.9871 instead of 0.9729. With two characters of context, the trigram model can capture language-specific patterns such as *-ção* (Portuguese) versus *-ción* (Spanish), or *sch* and *th*.

Cost. Both models are built in about 2 s on a CPU and need very little memory. The peak memory during construction is 0.42 MB for the bigram and 2.64 MB for the trigram. The trigram uses more memory because the number of possible contexts grows from $|V|$ to $|V|^2$. Classifying all 700 test texts takes less than 0.2 s.

Error analysis. As Figure 1 shows, almost all errors involve Portuguese or Spanish. The bigram model misclassifies 12 Portuguese texts (7 as Spanish) and 5 Spanish texts (as Portuguese). The trigram model reduces this to 6 Portuguese and 3 Spanish errors. Representative errors are mostly very short or contain little language-specific evidence:

- "`4.800 --`" (Portuguese, 8 characters), predicted as German by both n-gram models. It contains no letters at all.
- "*Decidimos,*" (Portuguese), predicted as Spanish. The word is spelled identically in both languages.
- "*to protect us*" (English), predicted as Spanish, and "*un monde de murs,*" (French), predicted as Spanish.
- "*Ándale! Ándale!*" in the Portuguese test set, predicted as German. This is a Spanish expression quoted inside a Portuguese talk.

1.3 Character-Level RNN Language Models

Five character-level language models are trained, one per language, using the suggested settings: embedding dimension 128, hidden dimension 128, 2 layers, Adam with learning rate 0.001, 10 epochs, batch size 32, and cross-entropy loss. The input is `<BOS>` followed by the characters of the text. The target is the next character, ending with `<EOS>`, and padded positions are ignored in the loss. For each language, the checkpoint with the lowest validation perplexity is kept.

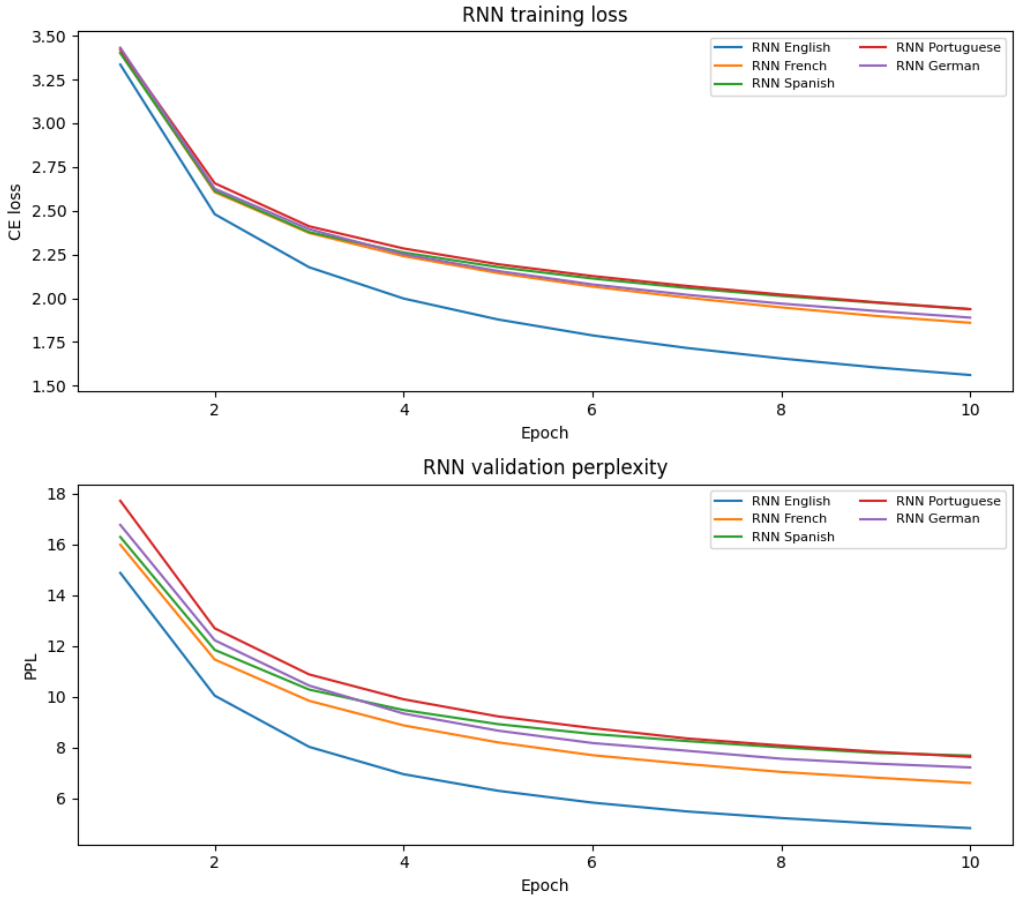


Figure 2: RNN training loss (top) and validation perplexity (bottom) for the five languages.

As Figure 2 shows, the training loss decreases steadily for all languages, falling from about 3.4 to between 1.56 and 1.94. The validation perplexity follows the same trend. Both curves are still decreasing at epoch 10, so the last epoch is selected for every language and no overfitting is observed. The best per-language validation perplexities are:

- English: 4.83
- French: 6.60
- German: 7.21
- Portuguese: 7.63
- Spanish: 7.68

English is the easiest to model because its spelling uses almost no diacritics.

Table 2: Test confusion matrix of the RNN models (rows: true language, columns: predicted language).

True \ Pred	English	French	Spanish	Portuguese	German
English	139	1	0	0	0
French	0	140	0	0	0
Spanish	0	1	136	3	0
Portuguese	0	2	2	135	1
German	0	0	0	0	140

Results. On `test.csv` the RNN models obtain:

- Accuracy: 0.9857
- Macro-F1: 0.9857
- Corpus-level test perplexity: 5.7054 (validation: 5.8937)

Cost. Training the five models takes 6.4 s in total on the GPU, with a peak GPU memory of 391.7 MB. Inference over the test set takes 4.1 s because every text must be passed through all five models.

Correct and incorrect predictions. Even very short texts are identified correctly when they contain language-specific function words or diacritics. Examples include "*Il est si*" (French, 9 characters), "*é preciosa,*" (Portuguese), and "*Y escalamos*" and "*Son una red*" (Spanish). As Table 2 shows, the 10 errors are concentrated on Spanish and Portuguese. These are short phrases made of words that are spelled the same or almost the same in both languages, which leaves too little evidence to separate them.

1.4 Character-Level LSTM Language Models

The LSTM models use exactly the same split, vocabulary, loss, optimiser, and hyperparameters as the RNN models. The only change is that the RNN layers are replaced with LSTM layers.

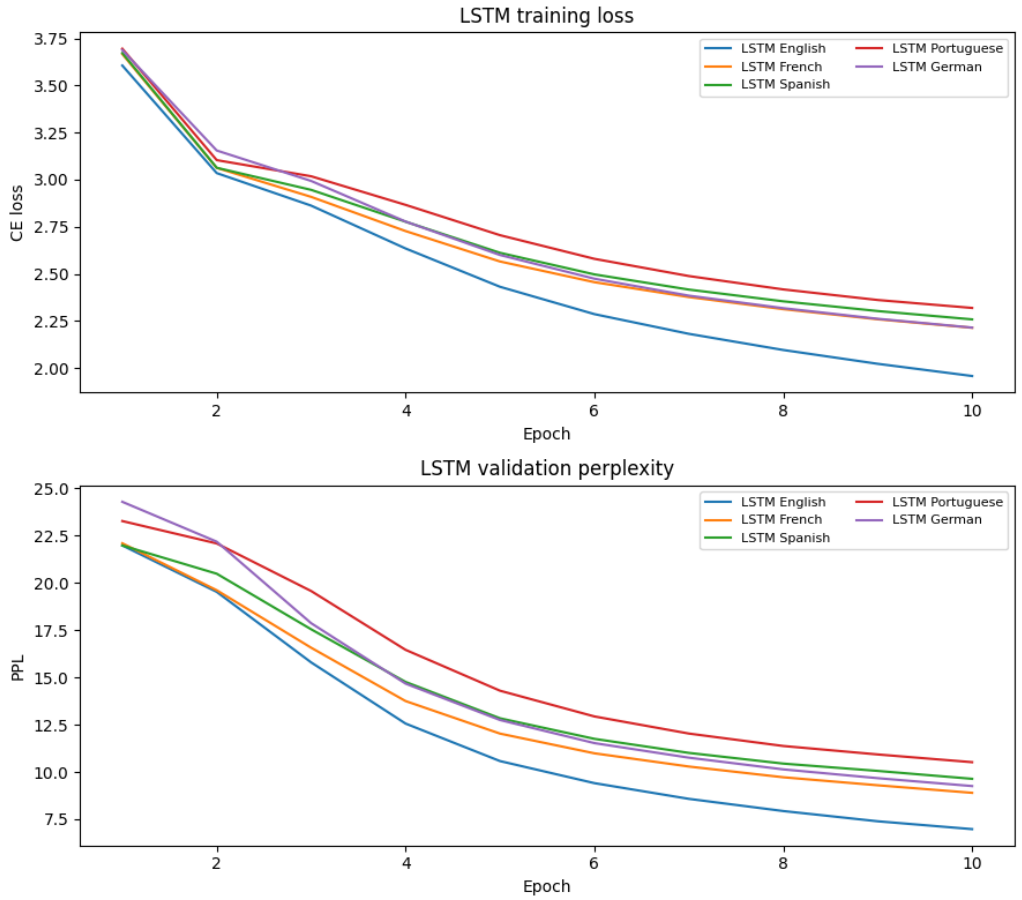


Figure 3: LSTM training loss (top) and validation perplexity (bottom) for the five languages.

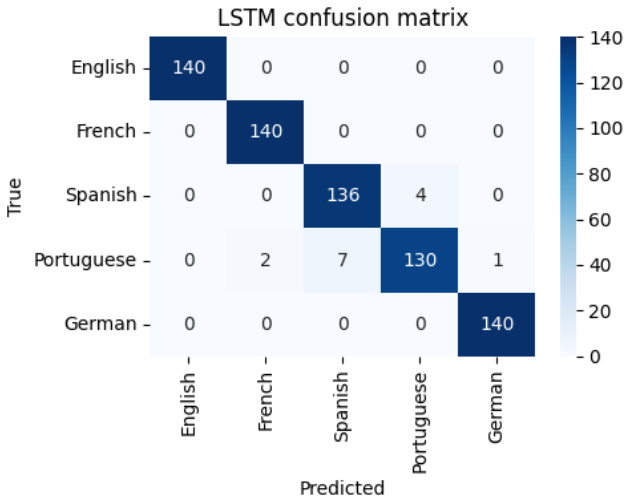


Figure 4: Test confusion matrix of the LSTM models.

Learning curves. The LSTM curves (Figure 3) also decrease monotonically, but they start higher and converge more slowly than the RNN curves. After 10 epochs, the training loss is between 1.96 and 2.32, and the best validation perplexities are:

- English: 6.98
- French: 8.90
- German: 9.25
- Spanish: 9.64
- Portuguese: 10.52

These are all higher than the RNN values. An LSTM has four gates per step and therefore about four times as many recurrent parameters, so the same 10-epoch budget leaves it further from convergence.

Results. On `test.csv` the LSTM models obtain:

- Accuracy: 0.9800
- Macro-F1: 0.9799
- Corpus-level test perplexity: 7.9834 (validation: 8.1334)

Cost. Training takes 9.0 s, with a peak GPU memory of 563.3 MB, and inference over the test set takes 5.1 s. Both are higher than for the RNN because of the larger recurrent cell.

Correct and incorrect predictions. Short texts such as "*Il est si*", "*Se comunican*", and "*as intuições*" are identified correctly. As Figure 4 shows, all 14 errors are Spanish or Portuguese texts: 7 Portuguese texts are predicted as Spanish, 4 Spanish texts as Portuguese, and the remaining 3 Portuguese texts as French or German.

1.5 Comparison and Discussion

Table 3: Comparison of the four model families on `test.csv`.

Model	Val PPL	Test PPL	Accuracy	Macro-F1	Train (s)	Inference (s)	Peak memory
Bigram	11.44	11.33	0.9729	0.9727	1.91	0.17	0.42 MB (CPU)
Trigram	9.70	9.40	**0.9871**	**0.9871**	2.12	0.18	2.64 MB (CPU)
RNN	**5.89**	**5.71**	0.9857	0.9857	6.42	4.08	391.7 MB (GPU)
LSTM	8.13	7.98	0.9800	0.9799	9.01	5.13	563.3 MB (GPU)

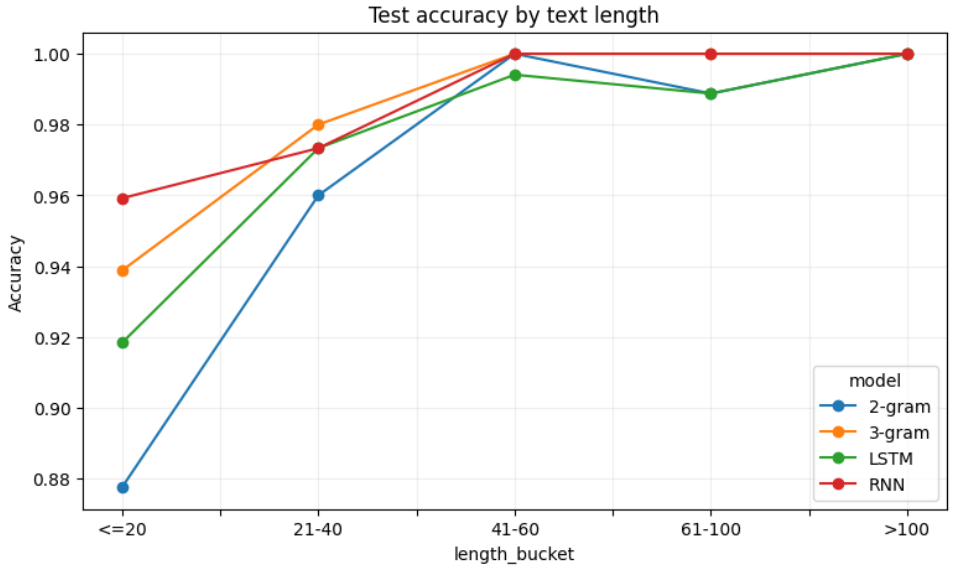


Figure 5: Test accuracy as a function of text length (number of characters).

Hardware and computation time. The n-gram models only need a CPU and a few megabytes of memory. They are built in about 2 s and classify the entire test set in under 0.2 s. The RNN and LSTM models need a GPU, with 392 MB and 563 MB of peak GPU memory respectively. They take longer to train, and their inference is more than 20 times slower, because every text requires a sequential forward pass through five neural models.

Does lower perplexity always mean higher accuracy? No. The RNN has by far the lowest perplexity (5.71, compared with 9.40 for the trigram), yet the trigram achieves the highest accuracy (0.9871 versus 0.9857). Likewise, the LSTM has lower perplexity than the trigram but lower accuracy. Corpus perplexity measures how well the correct language model predicts its own texts, averaged over all characters. Identification, however, depends only on the relative ranking of the five models for each individual text. A model can model long texts very well and still rank the wrong language first on a few short, ambiguous texts. Lower perplexity therefore tends to help, but it does not guarantee better identification accuracy.

Most frequently confused languages. Summed over all four models, the most frequent directed confusions are:

- Portuguese → Spanish: 19
- Spanish → Portuguese: 14
- Portuguese → French: 7
- Portuguese → German: 6

Spanish and Portuguese are by far the most confusable pair, because they share most of their vocabulary and character patterns. English, French, and German are almost never confused with one another.

Are shorter texts harder? Yes. As Figure 5 shows, every model performs worst on the shortest texts (at most 20 characters, 49 test samples). In that group, accuracy is 0.878 for the bigram, 0.918 for the LSTM, 0.939 for the trigram, and 0.959 for the RNN. Accuracy then rises with length:

- 21–40 characters: 0.960–0.980
- 41 characters or more: 0.989–1.000

Short texts provide only a few characters of evidence, and a single number, name, or shared word can dominate the score. Longer texts accumulate enough language-specific patterns to be classified almost perfectly. The RNN is the most robust model on short texts, while the bigram is the least robust.

2 Self-Attention Layer for Semantic Similarity Scoring

2.1 Model Design

Each example is converted into a single token sequence

$$z = \texttt{<context>}\ \text{context}\ \texttt{<sep>}\ \text{anchor}\ \texttt{<sep>}\ \text{target},$$

using the provided tokenizer and the vocabulary built from `train.csv` (`MAX_LEN` $= 64$, $d_{\text{model}} = 128$, 2 encoder layers, feed-forward size 256, batch size 64, seed 42).

The multi-head self-attention layer was implemented from scratch; the built-in multi-head attention module of PyTorch (`torch.nn.MultiheadAttention`) is not used. For each head,

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \qquad d_k = \frac{d_{\text{model}}}{h},$$

where Q, K, and V are linear projections of the input and M sets the scores of padded key positions to -10^9. The heads are concatenated and passed through an output projection. Each encoder layer uses post-layer normalisation:

$$x \leftarrow \text{LN}\big(x + \text{Drop}(\text{MHA}(x))\big), \qquad x \leftarrow \text{LN}\big(x + \text{Drop}(\text{FFN}(x))\big),$$

with FFN = Linear $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Linear. The input to the encoder is the scaled token embedding plus a sinusoidal positional encoding. The baseline uses masked mean pooling,

$$h_{\text{pool}} = \frac{\sum_i m_i h_i}{\sum_i m_i}, \qquad \hat{s} = \sigma(W h_{\text{pool}} + b),$$

and is trained with the MSE loss. Model selection uses the epoch with the highest validation Pearson correlation.

2.2 Learning Curves with 4 Heads

The baseline model uses 4 heads, masked mean pooling, AdamW (learning rate 10^{-3}, weight decay 10^{-4}), a ReduceLROnPlateau scheduler, and 10 training epochs.

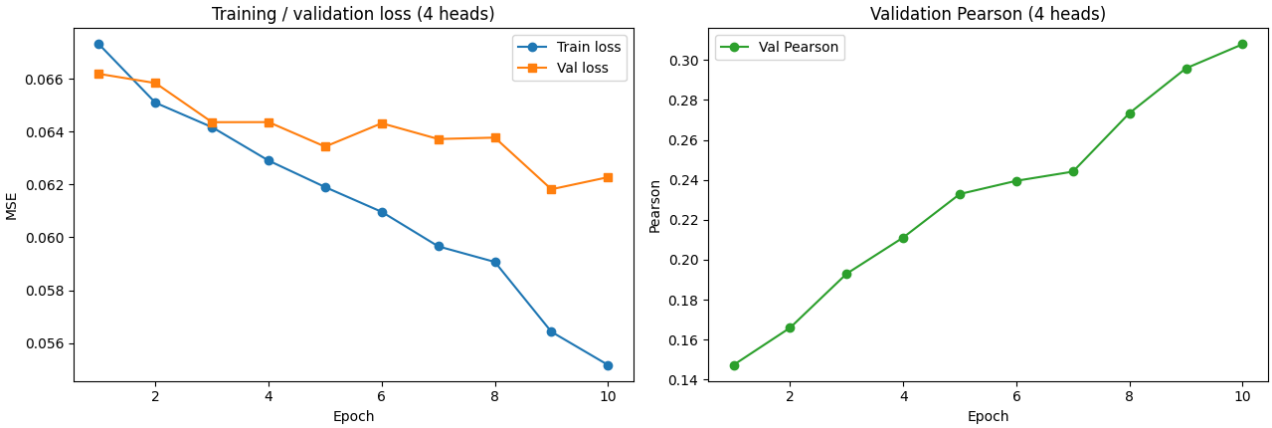


Figure 6: Baseline model with 4 heads. Left: training and validation MSE loss. Right: validation Pearson correlation.

As shown in Figure 6, the training loss decreases steadily from 0.0673 to 0.0552 over 10 epochs. The validation loss decreases much more slowly, from 0.0662 to 0.0618 at epoch 9, and rises slightly to 0.0623 at epoch 10. After about epoch 7, the training loss falls much faster than the validation loss, which is an early sign of overfitting.

The validation Pearson correlation rises steadily from 0.147 to 0.308 and is still increasing at epoch 10. Under the default 10-epoch budget, the model has therefore not converged in terms of ranking quality. The reduction in validation MSE is small, which suggests that the baseline training setup and plain mean pooling limit performance more than the model capacity does.

2.3 Validation Performance

Using the checkpoint with the best validation Pearson correlation (epoch 10), the baseline 4-head model obtains

$$\text{Pearson} = 0.3079, \qquad \text{RMSE} = 0.2496$$

on `val.csv`.

2.4 Effect of the Number of Attention Heads

The number of heads was set to 1, 4, and 8 while the data split, random seed, optimiser, scheduler, and 10-epoch budget were kept fixed.

Table 4: Validation performance for different numbers of attention heads.

Heads	Head dimension	Best val Pearson	Val RMSE	Training time (s)
1	128	**0.3104**	**0.2473**	34.5
4	32	0.3079	0.2496	36.0
8	16	0.2935	0.2491	38.0

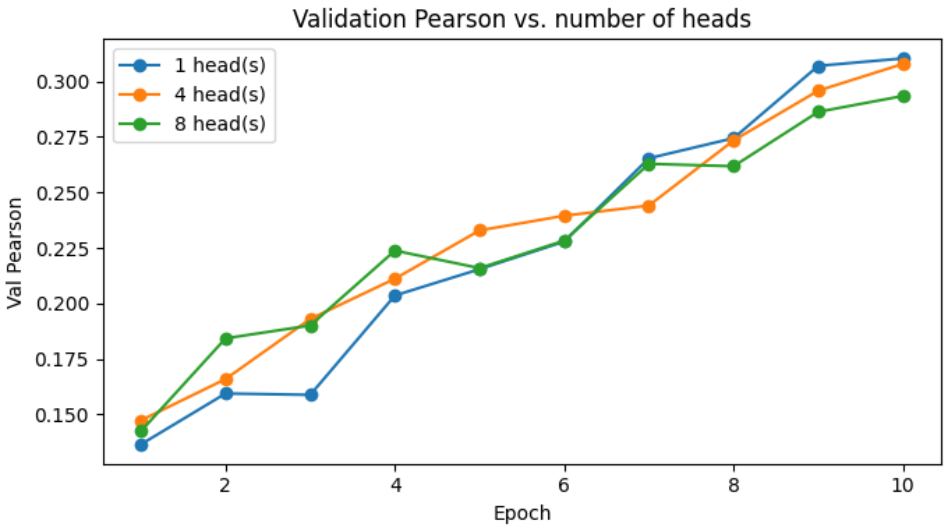


Figure 7: Validation Pearson correlation per epoch for 1, 4, and 8 attention heads.

All three settings have the same number of parameters, because the Q, K, V, and output projections are always 128×128. Only the way the 128-dimensional space is split into heads changes.

As Table 4 and Figure 7 show, the differences between the three settings are small (less than 0.02 Pearson), and the curves overlap for most epochs. The 1-head model is slightly best and the 8-head model is slightly worst. The inputs are very short (about 8 tokens on average), and the main signal is a word-level match between the anchor and the target. A single head with 128-dimensional queries and keys has a richer similarity function. With 8 heads, each head works in only 16 dimensions, which weakens its attention scores. More heads also make training slightly slower (34.5 s to 38.0 s).

All curves are still rising at epoch 10, so this comparison mainly reflects how quickly each setting converges under the same budget, not its final capacity. Overall, the number of heads is not the bottleneck for this task, and the 4-head setting is kept for the later experiments.

2.5 Hyperparameter Tuning

Starting from the baseline (T0), each configuration adds one documented change to the previous one. The model size, batch size, seed, and MSE loss are unchanged throughout. Every configuration is trained from the same seed, and the configuration with the highest validation Pearson correlation is selected.

Table 5: Tuning experiments. Each row adds the listed change to the previous row.

ID	Change	Val Pearson	Val RMSE	Best epoch
T0	Baseline (mean pooling, ReduceLROnPlateau, 10 epochs)	0.3079	0.2496	10
T1	Linear warm-up (6%) and cosine decay, up to 25 epochs with early stopping (patience 6), weight decay 0.01, gradient clipping 1.0, embedding initialisation std $d^{-1/2}$	0.5066	0.2399	25
T2	+ segment embeddings (context / anchor / target)	0.5042	0.2411	22
T3	+ interaction pooling $[g, u, v, \lvert u - v \rvert, u \odot v]$ with an MLP head	**0.6206**	0.2342	19
T4	+ token dropout 0.1 (words replaced by `<unk>`)	0.6060	0.2340	22
T5	T4 with dropout 0.1	0.6056	0.2213	24
T6	T4 with learning rate 5×10^{-4}	0.5992	0.2350	22

T1: schedule, budget, and initialisation (+0.199). This is the largest single improvement. The baseline initialises the token embeddings from $N(0, 1)$ and multiplies them by $\sqrt{128} \approx 11.3$, so the positional encoding, whose values lie in $[-1, 1]$, is almost invisible. Rescaling the initialisation to standard deviation $d^{-1/2}$ restores the positional information. The warm-up and cosine schedule,

together with the longer budget, also let the model keep improving instead of stopping while the Pearson correlation is still rising. Because T1 changes several settings at once, their individual contributions cannot be separated.

T2: segment embeddings (-0.002). The `<sep>` tokens and the positional encoding already indicate where each phrase lies, so segment embeddings add almost nothing.

T3: interaction pooling ($+0.116$). Mean pooling over the whole sequence mixes the anchor and target together. Interaction pooling computes the masked mean over the whole sequence (g), over the anchor tokens (u), and over the target tokens (v), and feeds $[g, u, v, |u - v|, u \odot v]$ to an MLP head. This gives the regressor explicit difference and alignment features between the two phrases, which suits a similarity task.

T4–T6: no further gain. Token dropout lowers the Pearson correlation by about 0.015. Every anchor in the validation and test sets also appears in `train.csv`, so remembering exact word identities is useful, and replacing words with `<unk>` removes that information. Lowering the dropout (T5) or the learning rate (T6) on top of T4 does not help either. T5 has the lowest RMSE (0.2213) but a lower Pearson correlation. Since model selection is based on Pearson correlation, **T3 was selected** as the final configuration.

2.6 Final Model and Kaggle Submission

The final model is an ensemble of five models trained with the T3 configuration. The seed is not reset between members, so they differ only in initialisation, shuffling order, and dropout, all drawn from the same seed-42 random stream. Their sigmoid outputs are averaged.

Table 6: Validation Pearson correlation of each ensemble member.

Member	0	1	2	3	4	Ensemble
Val Pearson	0.6206	0.6226	0.6127	0.6096	0.6286	**0.6409**

Table 7: Summary of the main models on `val.csv`.

Model	Val Pearson	Val RMSE
Baseline (4 heads, 10 epochs)	0.3079	0.2496
Best single tuned model (T3)	0.6206	0.2342
Final ensemble ($\times 5$)	**0.6409**	**0.2233**

The individual members reach validation Pearson correlations between 0.6096 and 0.6286. The ensemble reaches 0.6409, which is 0.020 higher than the best single tuned model, because averaging reduces the variance of models trained from scratch. Overall, the validation Pearson correlation improved from 0.308 for the baseline to 0.641 for the final model.

The final ensemble was used to predict all 7,295 samples in `test.csv`. The IDs were taken directly from `test.csv` in their original order and match `sample_submission.csv`. The predicted scores have mean 0.417 and standard deviation 0.255 and cover the full range from 0 to 1. This submission obtained a Kaggle leaderboard score of **0.66983** (Figure 8a). A later submission improved the leaderboard score to **0.69829** (Figure 8b).



1	315511079		0.66983	2	2h

(a) Submission of the five-member ensemble described above.



4	315511079		0.69829	6	2d

(b) Best submission.

Figure 8: Kaggle leaderboard scores.